CS496 Blockchain Tutorials - Solved Answers

Tutorials 3.1, 3.2, and 3.3 - exam-ready explanations

Source tutorials solved	3.1 Blockchain Basics; 3.2 Ethereum; 3.3 Smart Contracts & DApps
Total questions covered	48 questions
Correction note	The tutorial typos 'Sohana/Shana' are solved as Solana, matching the course slide on Solana Proof of History.
Grade note	This covers every blockchain tutorial question found in the reuploaded zip. No guide can guarantee 100%, but this is the material to study for a tutorial-based final.

How to use this for the final

- Memorize the short exam idea under each answer first.
- Then understand the explanation well enough to rewrite it in your own words.
- For diagram questions, reproduce the arrow flow diagrams in simple form; the final usually cares more about correct sequence than artistic drawing.
- Use exact keywords: UTXO, hash pointer, Merkle root, proof-of-work target, EVM, gas, smart contract, validator, attestation, Proof of History, Tower BFT.

Tutorial 3.1 - Blockchain Basics

Q1. What is the main motivation and purpose for building Bitcoin? (Satoshi Paper)

Answer: Bitcoin was built to create a purely peer-to-peer electronic cash system: payments can move directly from one party to another without a bank or trusted financial institution. The key problem it solves is double spending, using digital signatures, a peer-to-peer network, timestamps, hashing, and proof-of-work to create a ledger that is extremely hard to rewrite.

Easy explanation / exam tip: Exam idea: Bitcoin = peer-to-peer cash + no intermediary + solves double spending by PoW blockchain.

Q2. What are the major complex adaptive properties of Bitcoin and blockchains in general?

Answer: Bitcoin behaves like a complex adaptive system because many independent nodes interact without a central controller and still produce global order. Its main properties are decentralization, self-organization, emergent consensus, adaptation through difficulty adjustment, incentives through rewards/fees, robustness to failures, and feedback loops between miners, users, price, hash power, and difficulty.

Easy explanation / exam tip: Easy explanation: no one commands Bitcoin, but the rules and incentives make the network organize itself.

Q3. What is the difference between decentralized and distributed systems?

Answer: A distributed system means computation/data are spread across multiple machines. A decentralized system means control and decision-making are not held by one central authority. A system can be distributed but still centralized if one company controls all nodes. Bitcoin is both distributed and decentralized: many nodes store/verify data, and no single node controls the ledger.

Easy explanation / exam tip: Do not mix them: distributed = where the machines are; decentralized = where control is.

Q4. Explain how decentralization is achieved in Bitcoin.

Answer: Bitcoin decentralization is achieved through a peer-to-peer network, open participation, independent full-node verification, proof-of-work mining, public rules, and the longest/heaviest valid chain rule. Every full node can check transactions and blocks for itself. Miners propose blocks, but nodes accept them only if they satisfy Bitcoin rules.

Easy explanation / exam tip: Important: miners do not have unlimited power; full nodes enforce validity.

Q5. Explain how Bitcoin works. Provide a general overview.

Answer: Users control private keys in wallets. To pay someone, a wallet creates a transaction spending previous unspent transaction outputs (UTXOs), signs it, and broadcasts it to the peer-to-peer network. Nodes validate it and relay it. Miners collect valid transactions into candidate blocks and search for a nonce that makes the block hash below the target. The winning miner broadcasts the block. Other nodes verify it and append it to their copy of the blockchain. More blocks after it mean stronger confirmation.

Easy explanation / exam tip: Flow: wallet signs -> network validates -> miners mine -> nodes verify block -> ledger updates.

Q6. What are cryptographic hash functions, and what is their role in Bitcoin and blockchain?

Answer: A cryptographic hash function maps any input to a fixed-size output called a hash. It is deterministic, fast to compute, one-way, collision resistant, and highly sensitive to input changes. In Bitcoin it links blocks, summarizes transactions in the Merkle root, supports proof-of-work, identifies data, and makes tampering obvious because even a tiny change produces a different hash.

Easy explanation / exam tip: Bitcoin mainly uses SHA-256; changing a transaction changes the Merkle root, block header hash, and chain validity.

Q7. What is the role of public key cryptography in Bitcoin?

Answer: Public key cryptography gives users ownership and authorization. A user keeps a private key secret and derives a public key/address from it. To spend bitcoin, the user signs the transaction with the private key. Other nodes use the public key to verify the signature without learning the private key.

Easy explanation / exam tip: Private key spends. Public key verifies. Address receives.

Q8. How digital signatures are performed using public key cryptography?

Answer: The sender hashes the transaction data, signs that hash with their private key, and includes the signature and public key in the transaction. Validators recompute the transaction hash and use the public key to verify that the signature was produced by the matching private key and that the transaction was not changed after signing.

Easy explanation / exam tip: Signature proves authorization and integrity: right owner, unchanged message.

Q9. Describe the general structure of the Bitcoin ledger.

Answer: The Bitcoin ledger is an append-only chain of blocks. Each block contains a header and a list of transactions. The header includes the previous block hash and the Merkle root of current transactions. The ledger records transaction history through UTXOs rather than account balances. A user's spendable balance is the sum of UTXOs controlled by their keys.

Easy explanation / exam tip: Bitcoin ledger is not a table of accounts; it is a chain of transactions and unspent outputs.

Q10. What are the links between blocks in the ledger? How are the values of these links computed?

Answer: Each block links to the previous block by storing the hash of the previous block header. This hash is computed by applying SHA-256 twice to the previous block header. If an old block is modified, its hash changes, breaking the link to the next block and invalidating all later proof-of-work unless it is redone.

Easy explanation / exam tip: The link is a hash pointer: it points to data and proves the data was not changed.

Q11. What is a Merkle tree? And what is the significance of this tree in Bitcoin?

Answer: A Merkle tree is a binary hash tree that combines transaction hashes pair by pair until one final hash remains: the Merkle root. Bitcoin stores the Merkle root in the block header. This lets a block commit to all its transactions with one hash and allows efficient proof that a transaction is included in a block without downloading every transaction.

Easy explanation / exam tip: Merkle root = compact fingerprint of all transactions in the block.

Q12. Describe how a Merkle trees is built and illustrate how this tree can be used to prove the inclusion of a transaction in a block.

Answer: First hash every transaction: H(T1), H(T2), H(T3), H(T4). Then pair them: H12 = H(H(T1) || H(T2)) and H34 = H(H(T3) || H(T4)). Then compute the root: Hroot = H(H12 || H34). To prove T2 is included, provide T2, H(T1), H34, and the Merkle root. The verifier computes H(T2), combines it with H(T1) to get H12, combines H12 with H34 to get Hroot, and checks it matches the root in the block header.

Easy explanation / exam tip: Only the sibling hashes on the path are needed; this is why SPV/light verification is efficient.

Q13. Describe the general structure of a Bitcoin transaction.

Answer: A Bitcoin transaction contains inputs and outputs. Each input points to a previous unspent output and includes an unlocking script/signature proving the spender can spend it. Each output contains an amount and a locking script specifying the conditions for future spending. A transaction also includes metadata such as version and locktime.

Easy explanation / exam tip: Inputs consume old UTXOs. Outputs create new UTXOs.

Q14. Explain using a simple example how payments are performed in Bitcoin. What is so peculiar about the Bitcoin payment process.

Answer: Example: Alice has one UTXO worth 1 BTC and wants to pay Bob 0.3 BTC. Her wallet creates a transaction with one input spending the 1 BTC UTXO and two outputs: 0.3 BTC to Bob and about 0.6999 BTC back

to Alice as change, with the small difference as a miner fee. The peculiar part is that Bitcoin does not subtract from an account balance. It consumes complete UTXOs and creates new outputs, like melting coins and making new coins.

Easy explanation / exam tip: Key phrase: Bitcoin uses UTXO, not account balances.

Q15. Explain how transactions are validated.

Answer: Nodes validate a transaction by checking that inputs refer to existing unspent outputs, signatures/scripts are valid, the spender has authorization, input value is at least output value, no input is double-spent, amounts are valid, and consensus rules such as size, locktime, and format are satisfied. Miners include only valid transactions, and full nodes reject invalid blocks even if mined.

Easy explanation / exam tip: Validation checks both ownership and money conservation.

Q16. Describe the script used to validate transactions in Bitcoin.

Answer: A common Bitcoin payment script is Pay-to-Public-Key-Hash (P2PKH). The locking script on the previous output is: OP_DUP OP_HASH160 <publicKeyHash> OP_EQUALVERIFY OP_CHECKSIG. The unlocking script in the spending input provides: <signature> <publicKey>. During validation, the public key is duplicated and hashed, compared with the expected publicKeyHash, then OP_CHECKSIG verifies the signature against the transaction.

Easy explanation / exam tip: This proves: the public key matches the address, and the signature matches the public key.

Q17. Describe the general structure of Bitcoin block.

Answer: A Bitcoin block contains a block header and a transaction list. The header includes version, previous block hash, Merkle root, timestamp, difficulty target bits, and nonce. The transaction list starts with a coinbase transaction that creates the block reward and fees for the miner, followed by normal transactions.

Easy explanation / exam tip: Header is what miners hash; transactions are summarized by the Merkle root.

Q18. What is mining? What is the main purpose of the mining process?

Answer: Mining is the process where miners build candidate blocks and repeatedly hash the block header with different nonces until the hash is below the difficulty target. Its purposes are to secure consensus, decide who appends the next block, make rewriting history expensive, regulate block timing, and introduce new bitcoin plus transaction-fee rewards.

Easy explanation / exam tip: Mining is not just creating coins; it is Bitcoin's security and consensus mechanism.

Q19. Describe the mining process and the challenge on which the process is based.

Answer: Miners collect valid transactions, create a coinbase transaction, build the Merkle root, fill the block header, and vary the nonce/extra nonce until double-SHA-256(header) is numerically less than the target. The target is adjusted every 2016 blocks so blocks remain about 10 minutes apart. Lower target means higher difficulty.

Easy explanation / exam tip: Challenge: find a valid nonce. Easy to verify, hard to find.

Q20. What is the Genesis Block in Bitcoin?

Answer: The Genesis Block is Bitcoin's first block, also called block 0. It was created by Satoshi Nakamoto in January 2009 and is hardcoded as the starting point of the blockchain. It has no real previous block, so its previous block hash is a special zero value. All later Bitcoin blocks ultimately trace back to it.

Easy explanation / exam tip: It is the root of Bitcoin's block history.

Tutorial 3.2 - Ethereum

Q1. What is Ethereum? Provide a practical functional definition.

Answer: Ethereum is a decentralized programmable blockchain platform. Practically, it is a global replicated state machine where users send transactions, smart contracts run on the Ethereum Virtual Machine, and the network updates shared state by consensus. ETH is the native currency used to pay gas and transfer value.

Easy explanation / exam tip: Bitcoin mainly transfers value; Ethereum transfers value and executes code.

Q2. The designers of Ethereum sometimes call it 'The Decentralized World Computer.' What are the characteristics that makes it qualify for this title?

Answer: Ethereum qualifies because smart contract code and state are replicated across many nodes, the EVM executes contract code deterministically, anyone can deploy and call applications, results are agreed through consensus, and no single server controls execution. It behaves like one shared computer whose memory is the blockchain state and whose CPU is the EVM execution performed by nodes.

Easy explanation / exam tip: World computer = shared global state + deterministic computation + decentralized consensus.

Q3. What is the difference between Ethereum and the traditional web?

Answer: Traditional web applications usually use a client-server model: a company controls backend servers, databases, authentication, and business logic. Ethereum DApps use smart contracts as backend logic, wallets for identity/signing, blockchain state as shared storage, and gas-paid transactions for state changes. Ethereum is more transparent and tamper-resistant, but slower, more expensive, and less private than a normal web backend.

Easy explanation / exam tip: Traditional web: trust a server. Ethereum: verify a shared blockchain.

Q4. Describe the Ethereum echo system (architecture).

Answer: Ethereum's ecosystem includes users/wallets, externally owned accounts, contract accounts, transactions/messages, smart contracts, the EVM, execution clients, consensus clients/validators, peer-to-peer networking, mempools, blockchain/state databases, JSON-RPC APIs, DApp frontends, developer tools such as Solidity/compilers, and ETH/gas economics.

Easy explanation / exam tip: Architecture view: frontend/wallet -> JSON-RPC node -> EVM/contracts -> blockchain state -> consensus network.

Q5. What is EVM?

Answer: The Ethereum Virtual Machine is the deterministic, sandboxed runtime that executes compiled smart contract bytecode. Every validating node can run the same EVM instructions and reach the same result. EVM execution is gas-metered, meaning every operation has a cost to prevent infinite computation and spam.

Easy explanation / exam tip: EVM = Ethereum's smart-contract execution engine.

Q6. What are the types of accounts in Ethereum and what are the differences between them?

Answer: Ethereum has two major account types. Externally Owned Accounts (EOAs) are controlled by private keys, have no code, and can initiate transactions. Contract Accounts are controlled by smart contract code, have code and storage, and cannot start transactions by themselves; they execute only when called by an EOA or another contract. Both can hold ETH and have addresses.

Easy explanation / exam tip: EOA = user key. Contract account = code at an address.

Q7. What will be the impact of a message sent from an external account to a contract account?

Answer: A message/transaction from an EOA to a contract account activates the contract's code in the EVM. Depending on the function called and provided data, it may read or change contract storage, transfer ETH/tokens, emit events, call other contracts, or revert if conditions fail.

Easy explanation / exam tip: EOA-to-contract means code execution, not just value transfer.

Q8. What will be the purpose of a message from an external account to another external account?

Answer: A message/transaction from one EOA to another EOA is mainly a value transfer, usually sending ETH. Since the receiver has no contract code, no smart contract logic is executed, except normal transaction validation and balance/state update.

Easy explanation / exam tip: EOA-to-EOA is the Ethereum equivalent of a simple payment.

Q9. Describe the general structure of an Ethereum transaction, describing each field in the transaction and explaining the purposes they serve.

Answer: An Ethereum transaction generally includes: nonce - sender's transaction counter preventing replay and ordering transactions; to - recipient address, or empty for contract creation; value - ETH amount to transfer; data/input - function call data or contract bytecode; gas limit - maximum gas the sender allows; gas pricing fields - legacy gasPrice or EIP-1559 maxFeePerGas and maxPriorityFeePerGas; chain ID - prevents replay across chains; signature fields v/r/s - prove authorization by the sender's private key. After execution, the transaction produces a receipt with status, gas used, logs/events, and contract address if created.

Easy explanation / exam tip: For exams, name the fields and say what each protects or pays for.

Q10. What are the major components of a full node in Ethereum?

Answer: In the course architecture, a full node includes an Ethereum client and a blockchain database. The client contains the EVM, memory pool, client process, and JSON-RPC API. The blockchain database stores blocks, transactions, contract bytecode, account state, and contract storage. In modern post-Merge Ethereum, a full validating setup separates this into an execution client, consensus/beacon client, and validator client when staking.

Easy explanation / exam tip: Course answer: client + EVM + mempool + process + JSON-RPC + blockchain database.

Q11. What are the major types of nodes in Ethereum?

Answer: The course slides emphasize full nodes and mining nodes for the PoW-era architecture. A complete answer can also mention light nodes and archive nodes. In modern Ethereum, block production is done by validator nodes rather than miners. Full nodes verify state and blocks, archive nodes keep historical state, light nodes rely on proofs/headers, and validator nodes propose/attest blocks after staking.

Easy explanation / exam tip: Use both terms if needed: historical mining nodes; current validator nodes.

Q12. Using simple diagrams, describe the journey of a transaction from the time it enters the system until it finishes.

Answer: Journey: User/DApp creates and signs transaction -> JSON-RPC sends it to an Ethereum node -> execution client checks signature, nonce, balance, gas, and format -> valid transaction enters the mempool -> node gossips it to peers -> block producer selects it -> EVM executes it with other transactions -> new block is proposed -> other nodes re-execute and verify -> block is accepted/finalized -> transaction receipt is available and state is updated.

Easy explanation / exam tip: Diagram: Wallet/DApp -> Node/JSON-RPC -> Mempool -> Block Producer -> EVM Execution -> Block Broadcast -> Full Node Verification -> Final State.

Q13. Describe the Ethereum Proof of Work mining algorithm.

Answer: Ethereum's historical PoW algorithm was Ethash, derived from Dagger-Hashimoto. Miners used the block header, nonce, and a large DAG dataset to compute hashes until the result was below the difficulty target. Ethash was memory-hard to reduce ASIC advantage and encourage GPU mining. The DAG was generated from a cache and updated every epoch, making mining require large memory bandwidth. This is now historical because Ethereum moved from PoW to PoS.

Easy explanation / exam tip: Same core as Bitcoin: find nonce below target. Ethereum-specific detail: Ethash + DAG + memory-hard.

Q14. Describe Ethereum Proof of Stake algorithm.

Answer: Ethereum PoS uses validators instead of miners. A validator stakes ETH, runs execution/consensus/validator software, and can be selected pseudo-randomly to propose a block in a 12-second slot. Other validators in committees attest to valid blocks. LMD-GHOST chooses the head of the chain, and Casper FFG provides finality when at least two-thirds of stake supports checkpoints. Honest validators earn rewards; dishonest validators can be slashed.

Easy explanation / exam tip: PoS security comes from financial collateral, not electricity.

Q15. Describe the architecture of Ethereum.

Answer: Ethereum architecture can be described in layers: application layer with DApps and wallets; API layer using JSON-RPC; execution layer with EOAs, contract accounts, transactions, EVM, gas, and world state; data layer with blocks, receipts, logs, account/state tries; consensus layer with validators, attestations, fork choice, and finality; and peer-to-peer network layer for gossiping transactions and blocks.

Easy explanation / exam tip: One-line version: DApps call smart contracts through nodes; nodes execute EVM state changes and reach consensus on blocks.

Tutorial 3.3 - Smart Contracts, DApps, and Solana

Q1. What are smart contracts and how do they work? Provide a detailed definition of smart contract in the context of Ethereum.

Answer: A smart contract in Ethereum is program code deployed to a contract account on the blockchain. It has an address, bytecode, storage, balance, and functions. Users or other contracts call it through transactions/messages. The EVM executes its bytecode deterministically on every validating node, and the resulting state changes become part of the blockchain only if the transaction and block are valid. Smart contracts enforce rules automatically without relying on a central server.

Easy explanation / exam tip: Smart contract = on-chain code + state + deterministic execution + consensus-enforced results.

Q2. What are the major differences between ordinary Ethereum transactions and Smart Contracts?

Answer: An ordinary EOA-to-EOA transaction mainly transfers ETH and changes balances. A smart contract interaction sends data to a contract address and triggers code execution, which can update storage, emit events, call other contracts, mint/transfer tokens, or reject the transaction. Contract deployment is also a transaction, but its data field contains bytecode and the result is a new contract account.

Easy explanation / exam tip: Transaction is the message. Smart contract is the on-chain program that may react to the message.

Q3. How are smart contracts validated?

Answer: Smart contracts are validated by deterministic re-execution. Nodes verify the transaction signature, nonce, gas, and balance, then execute the contract bytecode in the EVM. The transaction is valid only if execution follows protocol rules and the block's resulting state root matches what nodes compute. If execution fails or reverts, state changes are undone, but gas for attempted execution is still consumed.

Easy explanation / exam tip: Validation means every node can independently recompute the same result.

Q4. What are the major steps for deploying a smart contract?

Answer: Steps: write contract code, usually in Solidity; compile it into EVM bytecode and ABI; prepare a contract-creation transaction with no 'to' address and bytecode in the data field; sign it with an EOA; pay gas and broadcast it; validators include it in a block; EVM executes constructor code; if successful, Ethereum creates a new contract address and stores the runtime bytecode/state; users interact with it through ABI-encoded function calls.

Easy explanation / exam tip: Deployment is just a special Ethereum transaction that creates code at a new address.

Q5. Using pseudocode provide a simple example of a typical smart contract.

Answer: Example pseudocode:

```
contract SimpleVault {
    owner = message.sender
    balance = 0

    function deposit(amount):
        require(amount > 0)
        balance = balance + amount

    function withdraw(amount):
        require(message.sender == owner)
        require(balance >= amount)
        balance = balance - amount
        send(owner, amount)
}
```

This contract stores funds, allows deposits, and only lets the owner withdraw if the balance is sufficient.

Easy explanation / exam tip: Typical contract pattern: state variables + access control + require checks + state update.

Q6. What are Decentralized Applications (DApps)?

Answer: DApps are applications whose core logic or state is handled by smart contracts on a blockchain rather than only by centralized servers. A DApp usually has a normal web/mobile frontend, a wallet connection such as MetaMask, smart contracts as backend logic, blockchain storage/state, and sometimes off-chain services for indexing or user interface performance.

Easy explanation / exam tip: DApp = normal app interface + blockchain smart-contract backend.

Q7. How DApps differ from ordinary applications?

Answer: Ordinary applications usually rely on centralized servers, databases, usernames/passwords, and administrators who can change backend logic/data. DApps rely on wallet signatures, smart contracts, blockchain consensus, transparent state, and gas-paid transactions. DApps are harder to censor and easier to audit, but they are slower, less private, more expensive for computation, and harder to change after deployment.

Easy explanation / exam tip: DApps trade speed/convenience for trust minimization and transparency.

Q8. What are emerging blockchain systems, and what is their major advantage over classical blockchain systems?

Answer: Emerging blockchain systems are newer-generation blockchains designed to overcome limits of early systems such as Bitcoin and early Ethereum. They use techniques such as proof-of-stake, proof-of-history, sharding, parallel execution, zero-knowledge proofs, interoperability bridges, and modular architectures. Their main advantages are higher throughput, lower fees, faster finality, better scalability, lower energy usage, and stronger interoperability.

Easy explanation / exam tip: Classical chains: secure but slow. Emerging chains: try to scale while keeping decentralization/security.

Q9. Describe the Solana blockchain and explain how it works.

Answer: Solana is a high-performance public blockchain for smart contracts, DApps, and digital assets. It uses stake-weighted validators and combines Proof of Stake with Proof of History. Proof of History acts as a cryptographic clock that timestamps and orders events before consensus, reducing communication overhead. A scheduled leader produces entries/blocks, transactions can be processed efficiently, validators verify and vote, and Tower BFT helps the network agree on the chain.

Easy explanation / exam tip: Course typo note: 'Sohana/Shana' means Solana.

Q10. Explain how blocks are validated in Solana.

Answer: In Solana, transactions are placed into a PoH sequence that proves their order. A leader/validator produces a block of entries using this sequence. Other validators verify the PoH hashes, check transaction signatures/accounts, execute transactions, confirm the resulting state, and cast stake-weighted votes using Tower BFT. When a supermajority of stake votes for the block/chain, the block becomes confirmed/finalized according to the network's consensus rules.

Easy explanation / exam tip: Validation uses both cryptographic ordering and stake-weighted validator votes.

Q11. Describe in detail of the Proof-of-Stake consensus algorithm used by Solana.

Answer: Solana's PoS is stake-weighted. SOL holders can delegate stake to validators. Validators with more delegated stake have more voting weight and greater chance/responsibility in the leader schedule. Validators produce blocks when selected, verify blocks from other leaders, and vote on forks. Rewards are distributed to validators and delegators. The network needs a supermajority, commonly described as at least two-thirds of stake, to agree on blocks and maintain safety.

Easy explanation / exam tip: Stake gives voting weight; delegation lets normal SOL holders support validators.

Q12. What is the Proof-of-History and what is the role it plays in the operation of Solana Consensus Algorithm?

Answer: Proof of History is Solana's cryptographic clock: a continuous sequence of hashes where each output becomes the next input. Because the sequence is sequential and verifiable, it proves that events happened in a specific order and that time passed between them. Its role is not to replace consensus by itself; it pre-orders transactions and timestamps events so validators need less communication to agree, allowing PoS/Tower BFT to finalize blocks faster.

Easy explanation / exam tip: PoH answers: 'what happened first?' PoS/Tower BFT answers: 'what do validators agree is valid?'

Q13. What the rate of transactions per second in Solana?

Answer: The course-expected number is up to about 65,000 transactions per second as a theoretical/benchmark capacity. In real live network conditions, actual TPS is usually lower and changes over time depending on demand, transaction type, and measurement method. For the final, write: Solana can theoretically support around 65,000 TPS.

Easy explanation / exam tip: Use the course number: 65,000 TPS.

Double-check coverage checklist

- Tutorial 3.1 - Blockchain Basics: 20 questions solved.
- Tutorial 3.2 - Ethereum: 15 questions solved.
- Tutorial 3.3 - Smart Contracts, DApps, and Solana: 13 questions solved.
- Total: 48 / 48 blockchain tutorial questions solved.

Important accuracy notes

- Ethereum PoW/Ethash is historical. Ethereum now uses proof-of-stake. The real Merge date was September 15, 2022; one course slide says Dec 2022, but the official Ethereum date is September 15, 2022.
- For Solana TPS, the course answer is 65,000 TPS as theoretical/benchmark capacity. Real-world TPS is variable, so use the course number for the final unless your instructor explicitly asks for live network TPS.
- For Solana, Proof of History is not a standalone replacement for consensus. It provides ordering/time; stake-weighted validators and Tower BFT handle agreement.